

- **Proyecto:** Ecommify
- **Unidad:** PostgreSQL: Diseño relacional avanzado
- **Tema:** Diseño conceptual, lógico y arquitectura híbrida con PostgreSQL y MongoDB
- **Dataset base:** Brazilian E-Commerce Public Dataset by Olist
- **Integrantes:** Roberto Jose Breuer Rodriguez – Luis Fernando Beltran Chantre
- **Fecha:** 2026-05-24

1. Resumen ejecutivo

El presente documento desarrolla el diseño conceptual y lógico del módulo transaccional de Ecommify a partir del dataset público de Olist, un conjunto de datos de comercio electrónico. La propuesta adopta una arquitectura híbrida en la que PostgreSQL gestiona las operaciones transaccionales críticas por su consistencia, capacidad de normalización y soporte para tipos avanzados como JSONB y arrays, mientras MongoDB almacena información documental flexible asociada al catálogo y a contenidos semiestructurados. El diseño relacional se propone en tercera forma normal para garantizar integridad, evitar redundancia y facilitar consultas eficientes sobre pedidos, pagos, inventario y entregas. La propuesta incorpora además una matriz de decisión tecnológica, un análisis CAP y una estrategia de separación de responsabilidades entre motores de base de datos. Como resultado, el modelo soporta tanto la operación diaria del negocio como futuras extensiones analíticas y de producto

2. Análisis de requisitos

a) Contexto

Ecommify se concibe como una plataforma de comercio electrónico cuyo núcleo operativo se basa en la gestión de pedidos, pagos, envíos, inventario y clientes. El dataset de Olist resulta adecuado porque permite observar un pedido desde varias dimensiones: estado de la orden, precio, costo de envío, ubicación del cliente, atributos del producto y reseñas posteriores a la compra. Esa estructura hace posible modelar un sistema híbrido realista, donde las entidades transaccionales se mantienen fuertemente consistentes y los datos variables se separan hacia un almacén documental. El resultado es un diseño útil para operación, escalabilidad y mantenimiento

b) Requisitos funcionales

El sistema debe permitir registrar clientes, productos, pedidos, ítems de pedido, pagos, estados de envío y reseñas. También debe conservar historial de cambios de estado y trazabilidad por pedido. Además, debe soportar consultas sobre volumen de ventas, tiempos de entrega, formas de pago, comportamiento por región y desempeño de vendedores. Finalmente, debe permitir almacenar atributos variables del catálogo sin alterar continuamente el esquema relacional.

c) Requisitos no funcionales

La solución debe garantizar integridad referencial, disponibilidad alta y trazabilidad de la información. El modelo debe ser escalable para manejar crecimiento en órdenes, productos y eventos de negocio. El diseño debe facilitar mantenimiento, control de cambios y evolución del modelo. También debe permitir consultas analíticas sobre el histórico de pedidos sin comprometer la operación transaccional.

d) Entidades principales

Las entidades principales del dominio son: cliente, dirección, pedido, ítem de pedido, pago, envío, producto, vendedor, reseña y geolocalización. En Olist, una orden puede contener múltiples ítems, y cada ítem puede ser atendido por un vendedor distinto. Esa característica obliga a modelar relaciones uno a muchos y muchos a muchos mediante tablas intermedias. La geolocalización se usa para análisis de distribución geográfica y optimización logística.

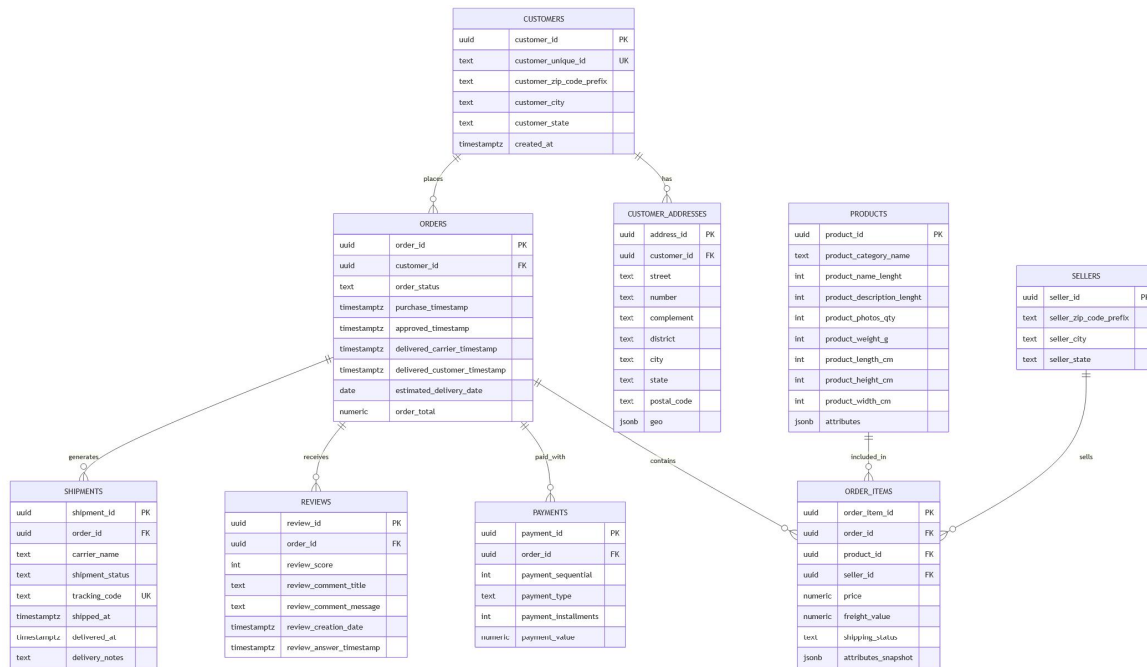
e) Tabla de volúmenes de datos esperados

Volumen esperado	Frecuencia de crecimiento	Observación
customers	Medio Alta	Crece con cada nuevo cliente registrado.
customer_addresses	Medio Alta	Puede haber más de una dirección por cliente.
orders	Alto	Es la tabla central del negocio y crece constantemente.
order_items	Muy alto	Cada pedido puede tener varios ítems, por eso acumula muchos registros.
payments	Alto	Un pedido puede tener uno o varios pagos.
products	Medio Baja	El catálogo crece más lento que las órdenes.
sellers	Bajo	La cantidad de vendedores suele ser menor y más estable.
reviews	Medio	Depende de cuántos pedidos sean evaluados por los clientes.
shipments	Alto	Cada pedido requiere trazabilidad de entrega.
geolocation	Baja	Suele ser una tabla muy grande, pero con poca

variación una vez cargada.

3. Diseño conceptual

a) Diagrama ER



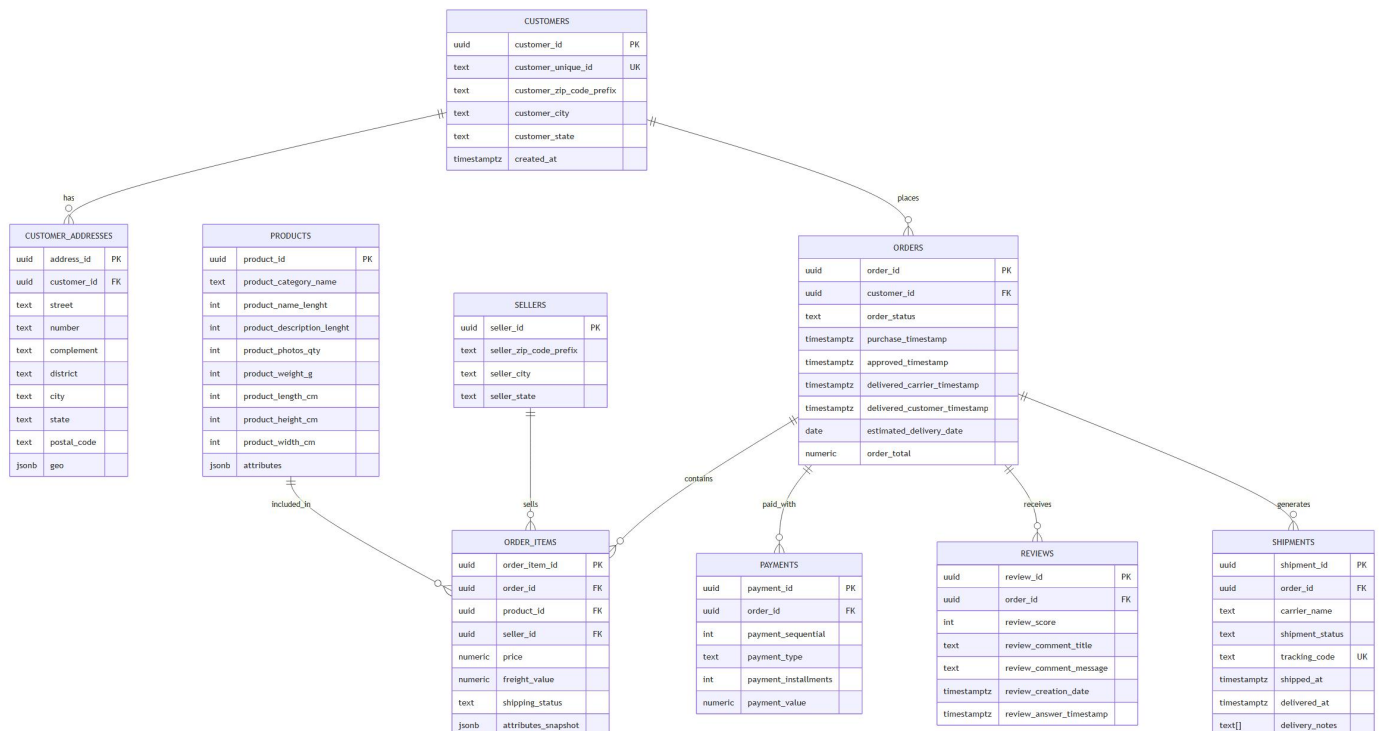
b) Entidades y relaciones

- Cliente se relaciona con una o varias direcciones.
- Cliente genera muchos pedidos.
- Pedido contiene uno o varios ítems de pedido.
- Ítem de pedido referencia un producto y un vendedor.
- Pedido tiene uno o varios pagos.
- Pedido puede generar una o varias reseñas.
- Pedido se vincula a una entrega o envío.
- Producto pertenece a una categoría y puede tener atributos variables.
- Vendedor despacha productos y participa en ítems de pedido.

c) Restricciones de negocio

- Un pedido debe tener al menos un ítem.
- Un pedido debe asociarse a un cliente existente.
- Un pago debe pertenecer a un pedido válido.
- Un ítem no puede existir sin pedido, producto y vendedor.
- Un pedido no debe marcarse como entregado si no tiene datos de envío.
- Las reseñas solo deben registrarse después de una compra.

4. Diseño lógico - Módulo PostgreSQL



Tipos avanzados justificados

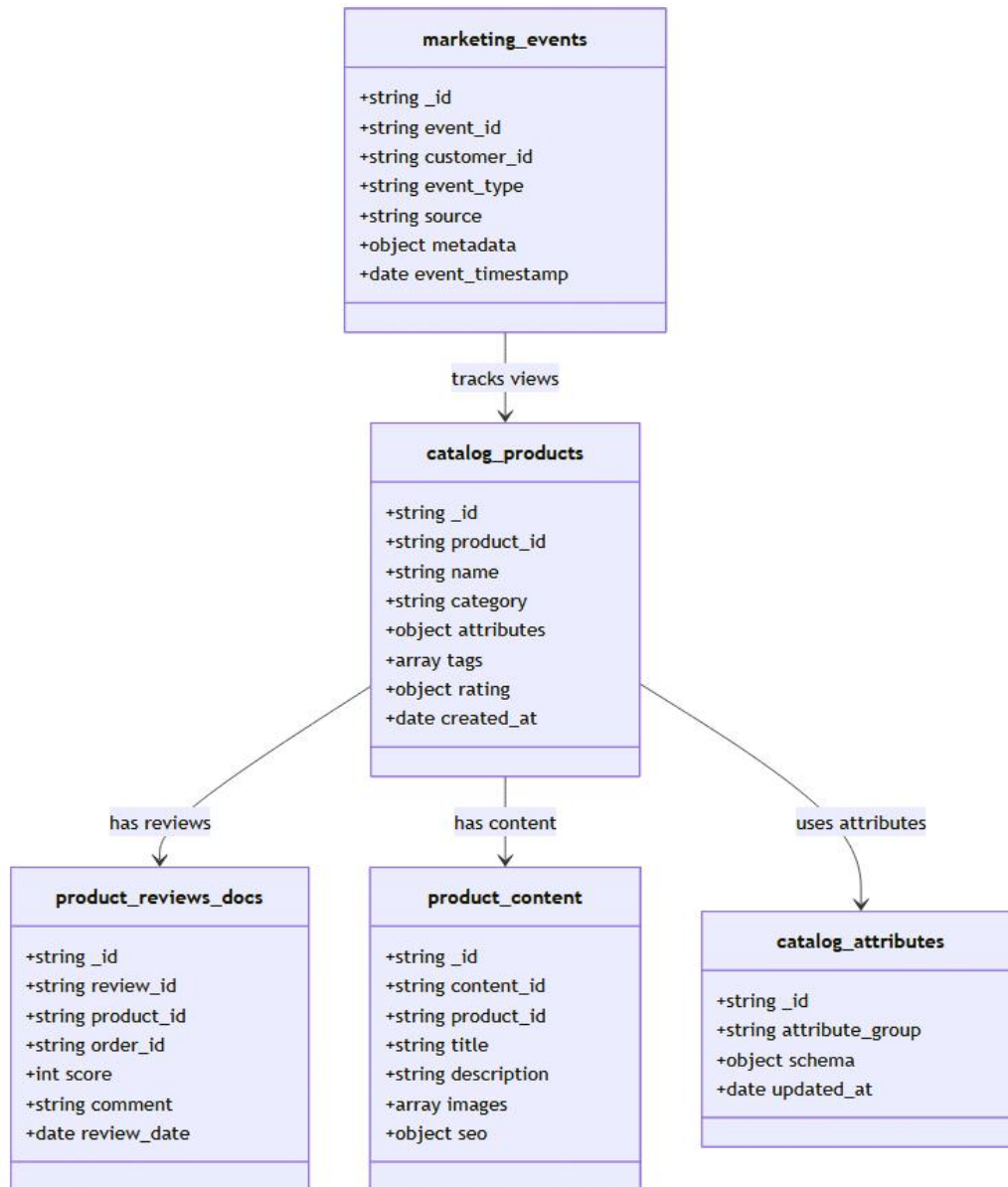
JSONB se usa para atributos flexibles de producto y snapshots de datos no completamente estables. Arrays (TEXT[]) se usan para notas o etiquetas múltiples. TIMESTAMPTZ es necesario para trazabilidad temporal con zona horaria. PostgreSQL soporta este tipo de extensibilidad y también la creación de tipos personalizados.

Extensiones PostgreSQL

Se recomienda usar:

- `pg_trgm` para búsquedas por similitud textual en nombres o comentarios.
- `pg_jsonschema` si desean validar estructuras JSONB.
- `uuid-oss` o `pgcrypto` para generación de UUID.
- `postgis` solo si van a explotar análisis geoespacial más avanzado.

5. Diseño lógico preliminar - Módulo MongoDB



Esquema de documentos

catalog_products

```
{
  "_id": "66f1a1d2e5b1a90001",
  "product_id": "P1001",
```

```
"name": "Tenis deportivos",
"category": "calzado",
"brand": "Ecommify",
"base_price": 129900,
"currency": "COP",
"status": "active",
"attributes": {
  "color": "blanco",
  "sizes": [38, 39, 40],
  "material": "malla"
},
"tags": ["deportivo", "urbano"],
"created_at": "2026-05-24T20:00:00Z"
}
```

product_reviews

```
{
  "_id": "66f1a1d2e5b1a90002",
  "review_id": "R5001",
  "product_id": "P1001",
  "order_id": "O9001",
  "customer_id": "C3001",
  "score": 5,
  "comment": "Excelente calidad y entrega rápida.",
  "review_date": "2026-05-24T20:10:00Z",
  "metadata": {
    "verified_purchase": true,
    "language": "es"
  }
}
```

```
}  
}
```

product_content

```
{  
  "_id": "66f1a1d2e5b1a90003",  
  "content_id": "CNT1001",  
  "product_id": "P1001",  
  "title": "Tenis deportivos para uso diario",  
  "description": "Calzado liviano, cómodo y resistente.",  
  "images": [  
    "https://cdn.ecommify.com/p1001-1.jpg",  
    "https://cdn.ecommify.com/p1001-2.jpg"  
  ],  
  "seo": {  
    "slug": "tenis-deportivos",  
    "keywords": ["tenis", "deportivo", "calzado"]  
  }  
}
```

product_attributes

```
{  
  "_id": "66f1a1d2e5b1a90004",  
  "attribute_group": "footwear",  
  "schema": {  
    "color": "string",  
    "sizes": "array<int>",  
    "material": "string",  
    "gender": "string"
```



```
    },  
    "updated_at": "2026-05-24T20:15:00Z"  
  }  
  
  customer_events  
  {  
    "_id": "66f1a1d2e5b1a90005",  
    "event_id": "EV2001",  
    "customer_id": "C3001",  
    "event_type": "view_product",  
    "product_id": "P1001",  
    "source": "web",  
    "metadata": {  
      "device": "mobile",  
      "country": "CO"  
    },  
    "event_timestamp": "2026-05-24T20:20:00Z"  
  }
```

Patrones de modelado a aplicar

Se proponen los siguientes patrones de modelado:

- **Embedding** para atributos del producto, SEO y metadatos pequeños dentro del mismo documento.
- **Referencing** para relacionar reseñas, eventos y contenido con product_id, order_id y customer_id.
- **Denormalización controlada** para repetir campos como nombre del producto o categoría en documentos de consulta frecuente.
- **Modelado orientado a acceso** para agrupar juntos los datos que suelen consultarse al mismo tiempo, como producto + contenido + atributos.

MongoDB recomienda diseñar según los patrones de acceso, almacenando juntos los datos que se consultan juntos y usando embedding o referencing según el caso.

Justificación MongoDB vs PostgreSQL

Los datos que van a **MongoDB** son aquellos con estructura flexible, evolución frecuente o necesidad de documentos completos: catálogo enriquecido, contenido de producto, reseñas semiestructuradas y eventos de comportamiento. MongoDB permite trabajar con documentos JSON/BSON anidados y adaptar el esquema con facilidad a medida que cambia el negocio.

Los datos que van a **PostgreSQL** son los transaccionales y fuertemente relacionados: pedidos, pagos, ítems de pedido, envíos, clientes y vendedores. Esos datos requieren integridad referencial, normalización y consistencia fuerte, por lo que PostgreSQL es mejor para el núcleo operativo.

6. Decisiones arquitectónicas justificadas

Matriz de decisión

Entidad / dato	Motor	Justificación
Pedidos	PostgreSQL	Requiere integridad fuerte, auditoría y transacciones ACID.
Ítems de pedido	PostgreSQL	Dependen del pedido y del inventario; no deben quedar inconsistentes.
Pagos	PostgreSQL	La consistencia es crítica para evitar cobros duplicados o estados inválidos.
Envíos	PostgreSQL	Necesitan trazabilidad exacta y estado único por orden.
Clientes	PostgreSQL	Datos estructurados, relacionales y con integridad referencial.
Productos base	PostgreSQL	Catálogo principal y llaves de relación con

Entidad / dato	Motor	Justificación
----------------	-------	---------------

pedidos e ítems.

Atributos variables del producto	MongoDB	Cambian con frecuencia y se benefician de esquema flexible.
----------------------------------	---------	---

Contenido enriquecido del producto	MongoDB	Descripciones, imágenes, SEO y metadatos semiestructurados.
------------------------------------	---------	---

Reseñas extensas / semiestructuradas	MongoDB	Mejor para texto libre, metadata y evolución del formato.
--------------------------------------	---------	---

Eventos de interacción	MongoDB	Datos de alta variabilidad y bajo acoplamiento transaccional.
------------------------	---------	---

7. Anexos

Diccionario de datos completo

Tabla / colección	Campo	Tipo	Descripción	Clave
customers	customer_id	UUID	Identificador del cliente.	PK
customers	customer_unique_id	TEXT	Identificador único de negocio.	UK
customers	customer_city	TEXT	Ciudad del	

Tabla / colección	Campo	Tipo	Descripción	Clave
			cliente.	
customers	customer_state	TEXT	Estado del cliente.	
orders	order_id	UUID	Identificador del pedido.	PK
orders	customer_id	UUID	Cliente que realiza el pedido.	FK
orders	order_status	TEXT	Estado del pedido.	
orders	purchase_timestamp	TIMESTAMPTZ	Fecha y hora de compra.	
order_items	order_item_id	UUID	Identificador del ítem.	PK
order_items	order_id	UUID	Pedido asociado.	FK
order_items	product_id	UUID	Producto comprado.	FK
order_items	seller_id	UUID	Vendedor asociado.	FK
payments	payment_id	UUID	Identificador del	PK

Tabla / colección	Campo	Tipo	Descripción	Clave
			pago.	
payments	order_id	UUID	Pedido pagado.	FK
payments	payment_type	TEXT	Tipo de pago.	
payments	payment_value	NUMERIC(12,2)	Valor pagado.	
products	product_id	UUID	Identificador del producto.	PK
products	product_category_name	TEXT	Categoría del producto.	
products	attributes	JSONB	Atributos variables del producto.	
sellers	seller_id	UUID	Identificador del vendedor.	PK
sellers	seller_city	TEXT	Ciudad del vendedor.	
reviews	review_id	UUID	Identificador de la reseña.	PK
reviews	order_id	UUID	Pedido evaluado.	FK

Tabla / colección	Campo	Tipo	Descripción	Clave
reviews	review_score	INT	Calificación de 1 a 5.	
shipments	shipment_id	UUID	Identificador del envío.	PK
shipments	order_id	UUID	Pedido enviado.	FK
shipments	tracking_code	TEXT	Código de seguimiento.	UK

Scripts SQL preliminares

```
CREATE TABLE customers (
  customer_id uuid PRIMARY KEY,
  customer_unique_id text UNIQUE NOT NULL,
  customer_zip_code_prefix text,
  customer_city text,
  customer_state text,
  created_at timestamptz DEFAULT now()
);
```

```
CREATE TABLE products (
  product_id uuid PRIMARY KEY,
  product_category_name text,
  product_name_length int,
```

```
product_description_length int,  
product_photos_qty int,  
product_weight_g int,  
product_length_cm int,  
product_height_cm int,  
product_width_cm int,  
attributes jsonb  
);
```

```
CREATE TABLE orders (  
  order_id uuid PRIMARY KEY,  
  customer_id uuid NOT NULL REFERENCES customers(customer_id),  
  order_status text NOT NULL,  
  purchase_timestamp timestamptz NOT NULL,  
  approved_timestamp timestamptz,  
  delivered_carrier_timestamp timestamptz,  
  delivered_customer_timestamp timestamptz,  
  estimated_delivery_date date  
);
```

```
CREATE TABLE order_items (  
  order_item_id uuid PRIMARY KEY,  
  order_id uuid NOT NULL REFERENCES orders(order_id),  
  product_id uuid NOT NULL REFERENCES products(product_id),  
  seller_id uuid NOT NULL,  
  price numeric(12,2) NOT NULL CHECK (price >= 0),  
  freight_value numeric(12,2) NOT NULL CHECK (freight_value >= 0),
```

```
attributes_snapshot jsonb
);
```

```
CREATE TABLE payments (
  payment_id uuid PRIMARY KEY,
  order_id uuid NOT NULL REFERENCES orders(order_id),
  payment_sequential int NOT NULL,
  payment_type text NOT NULL,
  payment_installments int NOT NULL CHECK (payment_installments > 0),
  payment_value numeric(12,2) NOT NULL CHECK (payment_value >= 0)
);
```

```
CREATE TABLE reviews (
  review_id uuid PRIMARY KEY,
  order_id uuid NOT NULL REFERENCES orders(order_id),
  review_score int NOT NULL CHECK (review_score BETWEEN 1 AND 5),
  review_comment_title text,
  review_comment_message text,
  review_creation_date timestampz,
  review_answer_timestamp timestampz
);
```

Ejemplos de datos

```
INSERT INTO customers (customer_id, customer_unique_id, customer_city,
customer_state)
VALUES ('11111111-1111-1111-1111-111111111111', 'CUST-001', 'Bogotá', 'BOG');
```


INSERT INTO products (product_id, product_category_name, attributes)

VALUES (

'22222222-2222-2222-2222-222222222222',

'calzado',

'{"color":"blanco","sizes":[38,39,40],"material":"malla"}':jsonb

);

INSERT INTO orders (order_id, customer_id, order_status, purchase_timestamp, estimated_delivery_date)

VALUES (

'33333333-3333-3333-3333-333333333333',

'11111111-1111-1111-1111-111111111111',

'delivered',

now(),

'2026-06-01'

);

Ejemplo MongoDB

{

"_id": "66f1a1d2e5b1a90001",

"product_id": "22222222-2222-2222-2222-222222222222",

"name": "Tenis deportivos",

"category": "calzado",

"attributes": {

"color": "blanco",

"sizes": [38, 39, 40],

"material": "malla"

```
},  
  "tags": ["deportivo", "urbano"],  
  "created_at": "2026-05-24T20:00:00Z"  
}
```